

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)



OCT 28, 2015 READ TIME: 9 MIN

[Apache Kafka](#) has a data structure called the “request purgatory”. The [purgatory](#) holds any request that hasn’t yet met its criteria to succeed but also hasn’t yet resulted in an error. The problem is “How can we efficiently keep track of tens of thousands of requests that are being asynchronously satisfied by other activity in the cluster?”

Kafka implements several request types that cannot immediately be answered with a response. Examples:

- A produce request with `acks=all` cannot be considered complete until all in-sync replicas have acknowledged the write and we can guarantee it will not be lost if the leader fails.
- A fetch request with `min.bytes=1` won’t be answered until there is at least one new byte of data for the consumer to consume. This allows a “long poll” so that the consumer need not busy wait checking for new data to arrive.

Table of Contents

1. [Old Purgatory Design](#)
2. [New Purgatory Design](#)
3. [Benchmark](#)
4. [Summary](#)
5. [References](#)

**Get started free with
Confluent**

[Get started](#)

**Watch demo: Kafka
streaming in 10 minutes**

[Watch now](#)

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)

[Login](#) [Contact Sales](#)

complete or (b) some timeout occurs.

The number of these asynchronous operations in flight at any time scales with the number of connections, which, for Kafka, is often tens of thousands.

The request purgatory is designed for such a large scale request handling, but the old implementation had a number of deficiencies.

In this blog, I would like to explain the problem with the old implementation and how the new implementation solved it. I will also present benchmark results.

Old Purgatory Design

The request purgatory consists of a timeout timer and a hash map of watcher lists for event driven processing. A request is put into the purgatory when it is not immediately satisfiable because of unmet conditions. A request in the purgatory is completed later when the conditions are met or is forced to be completed (timeout) when it passed beyond the time specified in the timeout parameter of the request. In the old design, it used Java *DelayQueue* to implement the timer.

When a request is completed, the request is not deleted from the timer or watcher lists immediately. Instead, completed requests are deleted as they were found during condition checking. When the deletion does not keep up, the server may exhaust JVM heap and cause *OutOfMemoryError*.

WRITTEN BY



Yasuhiro Matsuda

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)

requests from the purgatory when the number of requests (either pending or completed) in the purgatory exceeds the configured number. The purge operation scans the timer queue and all watcher lists to find completed requests and deletes them.

By setting this configuration parameter low, the server can virtually avoid the memory problem. However, the server must pay a significant performance penalty if it scans all lists too frequently.

New Purgatory Design

The goal of the new design is to allow immediate deletion of a completed request and reduce the load of expensive purge process significantly. It requires cross referencing of entries in the timer and the requests. Also it is strongly desired to have $O(1)$ insert/delete cost since insert/delete operation happens for each request/completion.

To satisfy these requirements, we designed a new purgatory implementation based on [Hierarchical Timing Wheels](#) [1].

Hierarchical Timing Wheel

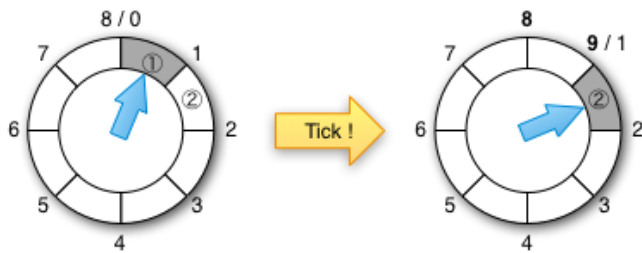
A simple timing wheel is a circular list of buckets of timer tasks. Let u be the time unit. A timing wheel with size n has n buckets and can hold timer tasks in $n * u$ time interval. Each bucket holds timer tasks that fall into the corresponding time range.

At the beginning, the first bucket holds tasks for $[0, u)$, the second bucket holds tasks for $[u, 2u)$, ..., the n -th bucket for $[u * (n - 1), u * n)$.

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)

[Login](#) [Contact Sales](#)

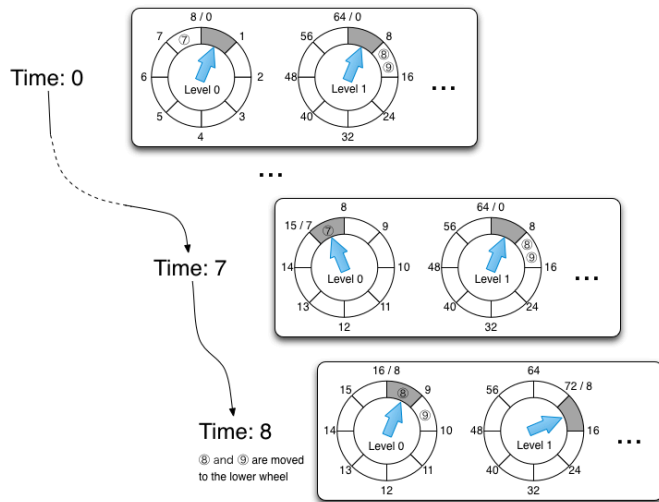
expire all timer tasks in it. So, the timer never inserts a task into the bucket for the current time since it is already expired. The timer immediately runs the expired task. The emptied bucket is then available for the next round, so if the current bucket is for the time t , it becomes the bucket for $[t + u * n, t + (n + 1) * u)$ after a tick. A timing wheel has $O(1)$ cost for insert/delete (start-timer/stop-timer) whereas priority queue based timers, such as `java.util.concurrent.DelayQueue` and `java.util.Timer`, have $O(\log n)$ insert/delete cost. Note that neither `DelayQueue` or `Timer` supports random delete.



A major drawback of a simple timing wheel is that it assumes that a timer request is within the time interval of $n * u$ from the current time. If a timer request is out of this interval, it is an overflow. A hierarchical timing wheel deals with such overflows. It is a hierarchically organized timing wheels that delegate overflows to upper level wheels. The lowest level has the finest time resolution. Time resolutions become coarser as we move up the hierarchy. If the resolution of a wheel at one level is u and the size is n , the resolutions should be $n * u$ in the second level, $n^2 * u$ in the third level, and so on. At each level overflows are delegated to the wheel in one level higher. When the wheel in the higher level ticks, it inserts timer tasks to the lower level. An overflow wheel can be created on-demand.

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)

timer recursively. The tasks are then moved to the finer grain wheels or be executed. The insert (start-timer) cost is $O(m)$ where m is the number of wheels, which is usually very small compared to the number of requests in the system, and the delete (stop-timer) cost is still $O(1)$.



Doubly Linked List for Buckets in Timing Wheels

In the new design, we use our own implementation of doubly linked list for the buckets in a timing wheel. The advantage of doubly linked list is that it allows $O(1)$ insert/delete of a list item if we have access to link cells in a list.

A timer task instance saves a link cell in itself when enqueued to a timer queue. When a task is completed or canceled, the list is updated using the link cell saved in the task itself.

Driving Clock using DelayQueue

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)

ticking, which checks if there is any task in the bucket. The unit time of the purgatory is 1ms ($u = 1\text{ms}$). This can be wasteful if requests are sparse at the wheel at the lowest level. This is usually the case because the majority of requests are satisfied before inserted into the wheel at the lowest level. It would be nice if a thread wakes up only when there is a non-empty bucket to expire. The new purgatory does so by using `java.util.concurrent.DelayQueue` similarly to the old implementation, but we enqueue task buckets instead of individual tasks. This design has a performance advantage. The number of items in `DelayQueue` is capped by the number of buckets, which is usually much smaller than the number of tasks, thus the number of offer/poll operations to the priority queue inside `DelayQueue` will be significantly smaller.

Purging Watcher Lists

In the old implementation, the purge operation of watcher lists is triggered by the total size of the watcher lists. The problem is that the watcher lists may exceed the threshold even when there isn't many requests to purge. When this happens it increases the CPU load a lot. Ideally, the purge operation should be triggered by the number of completed requests the watcher lists.

In the new design, a completed request is removed from the timer queue immediately with $O(1)$ cost. It means that the number of requests in the timer queue is the number of pending requests exactly at any time. So, if we know the total number of distinct

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)

and the numbers completed but still watched requests, we can avoid unnecessary purge operations. It is not trivial to keep track of the exact number of distinct requests in the purgatory because a request may or may not be watched. In the new design, we estimate the total number of requests in the purgatory rather than trying to maintain the exactly number.

The estimated number of requests are maintained as follows.

1. The estimated total number of requests, E , is incremented whenever a new request is watched.
2. Before starting the purge operation, we reset the estimated total number of requests to the size of timer queue. This is the current number of pending requests. If no requests are added to the purgatory during purge, E is the correct number of requests after purge.
3. If some requests are added to the purgatory during purge, E is incremented to $E + \text{the number of newly watched requests}$. This may be an overestimation because it is possible that some of the new requests are completed and remove from the watcher lists during the purge operation. We expect the chance of overestimation and an amount of overestimation are small.

Benchmark

We compared the enqueue performance of purgatory implementations, the old implementation and the new implementation.

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)

purgatory was separated from the rest of the system and also uses a fake request which does nothing useful. So, the throughput of the purgatory in a real system may be lower than the number shown by the test.

In the test, the intervals of the requests are assumed to follow the exponential distribution. Each request takes a time drawn from a log-normal distribution. By adjusting the shape of the log-normal distribution, we can test different timeout rate.

The tick size is 1ms and the wheel size is 20. The timeout was set to 200ms. The data size of a request was 100 bytes. For a low timeout rate case, we chose 75percentile = 60ms and 50percentile = 20. And for a high timeout rate case, we chose 75percentile = 400ms and 50percentile = 200ms. Total 1 million requests are enqueued in each run.

Requests are actively completed by a separate thread. Requests that are supposed to be completed before timeout are enqueued to another *DelayQueue*. And a separate thread keeps polling and completes them. There is no guarantee of accuracy in terms of actual completion time.

The JVM heap size is set to 200m to reproduce a memory tight situation.

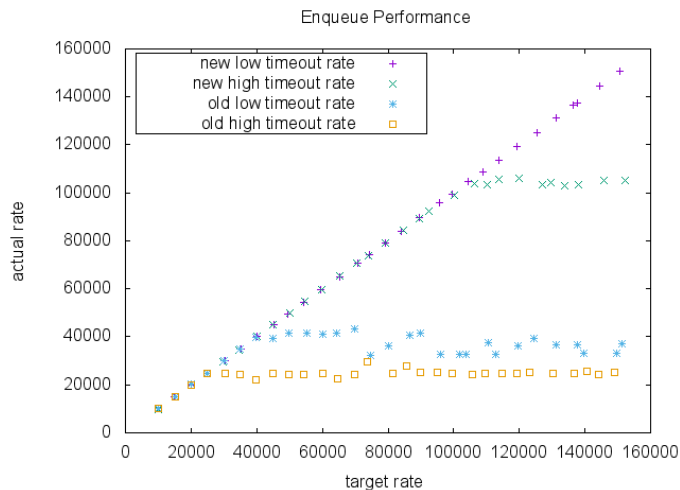
The result shows a dramatic difference in a high enqueue rate area. As the target rate increases, both implementations keep up with the requests initially. However, in low enqueue rate scenario the old implementation was saturated around 40000 RPS (request per



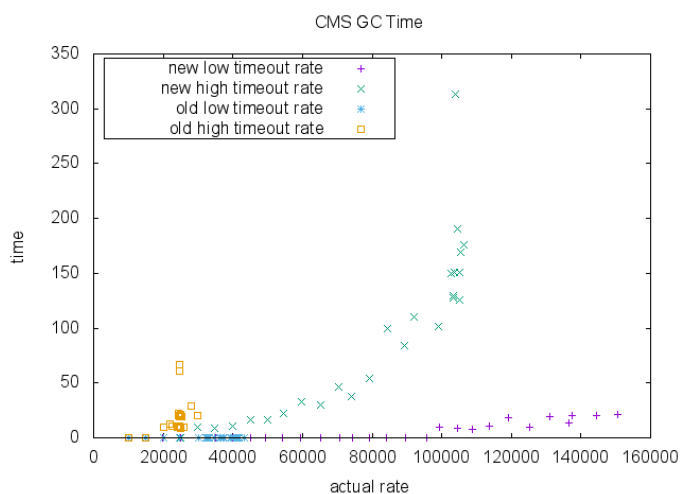
Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)

[Login](#) [Contact Sales](#)

degradation, and in high timeout scenario the old implementation was saturated around 25000 RPS, whereas the new implementation was saturated 105000 RPS in this benchmark.



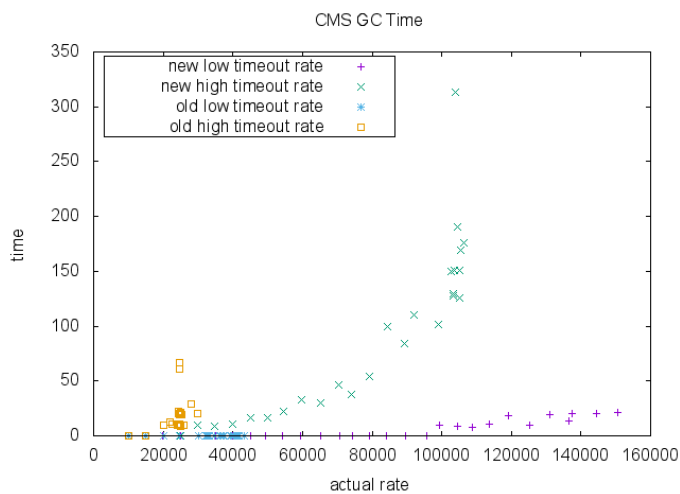
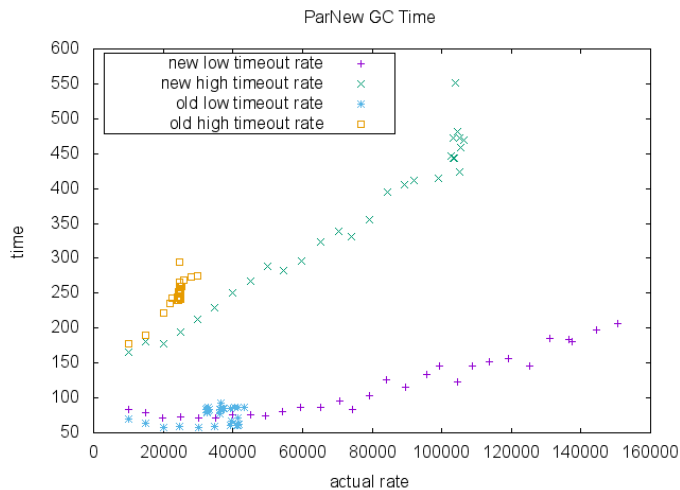
Also, CPU usage is significantly better in the new implementation. Note that the old implementation does not have data point higher than ~40000 RPS due to its scalability limit. Also notice that its CPU time saturates around 1.2 while it is steadily going up in the new implementation. It indicates that the old implementation may be hitting a concurrency issue due to synchronizations.



..., we measured total GC time (milliseconds) for *ParNew* collection and

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)

implementation in the region of enqueue rate that the old implementation can sustain. Again note that old implementation does not have data point higher than ~40000 RPS due to its scalability limit.



Summary

In the new design, we use Hierarchical Timing Wheels for the timeout timer and *DelayQueue* of timer buckets to advance the clock on demand. Completed requests are removed from the timer queue immediately with $O(1)$ cost. The buckets remain in the queue, however, the number of buckets is bounded. And, in a healthy system, most of the requests are satisfied

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)

delay queue. Thus, the timer should rarely have the buckets of the lower interval. The advantage of this design is that the number of requests in the timer queue is the number of pending requests exactly at any time. This allows us to estimate the number of requests need to be purged. We can avoid unnecessary purge operation of the watcher lists. As the result we achieve a higher scalability in terms of request rate with much better CPU usage.

References

[1] [George Varghese , Anthony Lauck, Hashed and hierarchical timing wheels: efficient data structures for implementing a timer facility, IEEE/ACM Transactions on Networking \(TON\), v.5 n.6, p.824-834, Dec. 1997](#)



[Download Apache Kafka and Confluent Platform](#)



Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | **Secure Your Spot**[Login](#) [Contact Sales](#)[Technology](#) < [Apache Kafka](#)

Subscribe to the Confluent blog

[Subscribe](#)

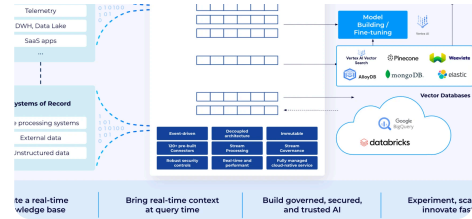
The True Cost of Real-Time Data Streaming

OCT 27, 2025

Explore the hidden costs of real-time streaming—compare infrastructure, ops, and ROI between Confluent Cloud and self...

[CONFLUENT STAFF](#)

Hands-on Workshop: ZooKeeper to KRaft Without the Hassle | [Secure Your Spot](#)
[Login](#) [Contact Sales](#)



Confluent: The Real-Time Backbone for Agentic Systems

OCT 16, 2025

Confluent, powered by Kafka, is the real-time backbone for agentic systems built with Google Cloud. It enables agents to access...

DUSTIN SHAMMO

MERLIN YAMSSI